

how Dahua, Hikvision or major cctv VMS companies visualize stream on the ts/js based UI? do they use webRTC? RTSP? MJPEG-over-HTTP?

Short answer: **almost never RTSP directly in the browser.**

Major VMS vendors (Dahua, Hikvision, Hanwha, Milestone, Genetec, Avigilon, Nx Witness, etc.) typically use **different streaming protocols depending on the scenario.**

Scenario	Typical Protocol	Why
Live view in browser	WebRTC or MSE (fMP4/H.264)	Low latency, browser compatible
Live view in native desktop VMS	RTSP or proprietary TCP	Full control, lowest overhead
Mobile app	WebRTC, RTSP, proprietary	Depends on architecture
Snapshot/preview	JPEG	Very lightweight
Recording playback	HLS or MSE	Seeking, buffering

First misconception

Many people think

```
Camera
  ↓
RTSP
  ↓
Browser
```

This is almost never true.

Chrome, Firefox, Safari...

do not support RTSP.

So there is always a server in between.

```
Camera
  RTSP
  ↓
Streaming Server
  ↓
WebRTC / MSE / HLS
  ↓
Browser
```

How Hikvision/Dahua usually work

Internally the camera still outputs

RTSP

or sometimes

Private SDK

The browser never sees RTSP.

Instead the VMS

1. connects RTSP
2. decodes if needed
3. repackages
4. sends browser-compatible stream

Option 1 — WebRTC (becoming the standard)

Modern VMS increasingly use

```
RTSP
  ↓
Gateway
  ↓
WebRTC
  ↓
Browser
```

Advantages

- 100–400 ms latency
- Hardware decode
- Audio
- Adaptive bitrate
- NAT traversal
- Browser native

Especially for

- cloud VMS
- AI dashboard
- remote monitoring

Option 2 — Media Source Extensions (MSE)

A lot of Chinese VMS use

```
RTSP
  ↓
ffmpeg
  ↓
Fragmented MP4
  ↓
Media Source Extension
```

Browser JavaScript receives

fMP4 fragments

and appends them into

MediaSource

Advantages

- Lower CPU
- Easier than WebRTC
- Can stay H.264
- Good browser compatibility

Latency

```
500 ms
to
2 seconds
```

Many enterprise VMS actually use this.

Option 3 — WebSocket + H264

Very common in surveillance.

Pipeline

RTSP

↓

Demux

↓

Raw H264 NAL Units

↓

WebSocket

↓

JavaScript Decoder

Decoder examples

- Broadway
- wasm-ffmpeg
- wasm-avc
- WebCodecs

This avoids transcoding.

Option 4 — WebCodecs (new)

Increasingly popular.

Pipeline

RTSP

↓

Extract H264 NAL

↓

WebSocket

↓

WebCodecs

↓

GPU decode

Latency

100–300 ms

CPU usage is much lower than JavaScript decoders.

Many new VMS are moving this direction.

Option 5 — MJPEG over HTTP

Older systems used

Camera

↓

MJPEG

↓

HTTP

↓

`<img src="">`

Advantages

- Very simple

Disadvantages

- Huge bandwidth
- No audio
- 10–15 fps
- Poor compression

Today mostly used only for

- thumbnails
- preview

- diagnostics

Rarely for a full VMS.

Desktop VMS is different

Desktop software like

- Milestone Systems
- Genetec
- Hanwha Vision
- Network Optix

often connects directly

Camera

↓

RTSP

↓

Native decoder

↓

GPU

No browser involved.

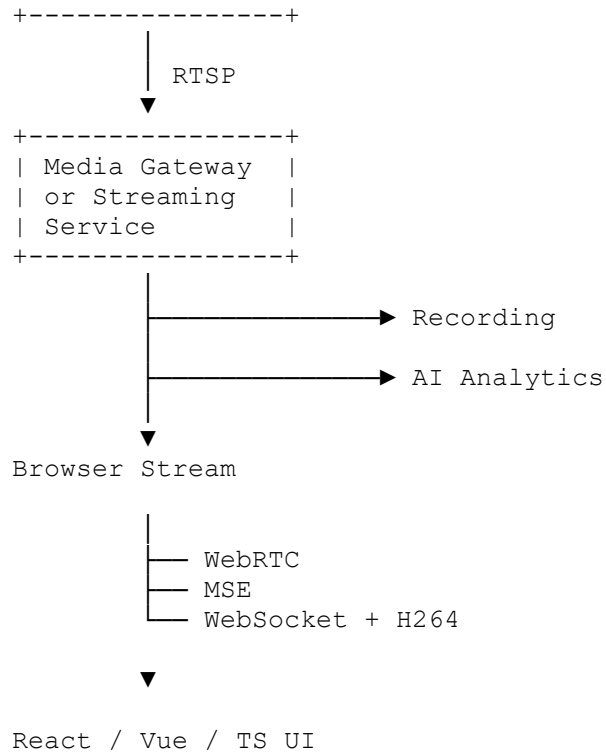
Latency can be

<100 ms

Dahua/Hikvision browser architecture (likely)

Although proprietary implementations differ, a common architecture is:

```
+-----+
| Camera |
| H264/H265 |
```



What about HLS?

Usually only for playback.

Camera
↓
RTSP
↓
HLS
↓
Browser

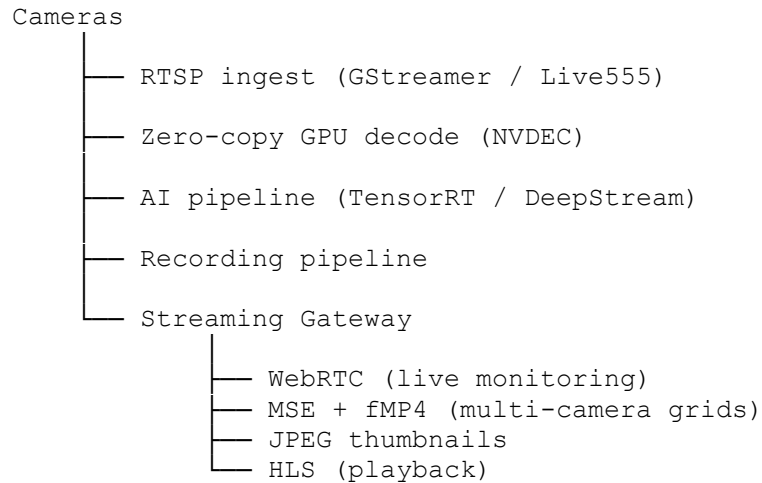
Latency

3-10 seconds

Terrible for security monitoring.

If I were building a modern AI VMS today

Given your background in real-time computer vision, GPU optimization, and your plan to build a reusable CV platform, I would separate the media plane from the AI plane.



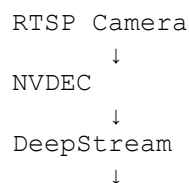
For the frontend (React/TypeScript), I would **not** expose RTSP directly. Instead:

- **WebRTC** for single-camera, ultra-low-latency viewing and PTZ interaction.
- **MSE + fMP4** for dashboards displaying many simultaneous camera feeds (e.g., 16–64 cameras), where scalability and browser efficiency matter more than achieving sub-200 ms latency.
- **HLS** only for archived playback and seeking.
- **MJPEG** only for lightweight previews or fallback scenarios.

This architecture matches how most modern enterprise VMS platforms are evolving while remaining compatible with browser capabilities and scalable AI backends.

and after AI inferencing with Deepstrea, when then stream as now bounding boses and AI overlays? short answer

Typically, **the AI overlays are burned into the video before it leaves the streaming server.**



Inference
↓
Bounding Boxes / Labels / OSD
↓
Encoder (H.264/H.265)
↓
WebRTC / MSE / WebSocket
↓
Browser

DeepStream usually uses **nvdssd (On-Screen Display)** to draw:

- Bounding boxes
- Class labels
- Track IDs
- Confidence
- Polygons
- Counting lines
- Heatmaps

before encoding the output stream.

More advanced VMS (recommended)

Instead of burning overlays into the video:

Video → Browser
|
Metadata (JSON/WebSocket)
↓
React/Canvas/WebGL overlays

This has several advantages:

- Crisp overlays at any zoom level
- User can toggle boxes on/off
- Easy object selection and interaction
- Lower bandwidth (video isn't re-encoded just to change overlays)
- Better UI for events, trajectories, alarms, etc.

For a modern AI VMS, I would recommend:

- **Video:** H.264/H.265 via WebRTC or MSE
- **Metadata:** WebSocket (or WebRTC DataChannel)
- **Overlay rendering:** HTML Canvas or WebGL in the React/TypeScript frontend

This is the architecture used by many newer AI surveillance platforms because it cleanly separates the media stream from AI metadata.

Thursday 11:23 PM

You said, WebRTC for single-camera, ultra-low-latency viewing and PTZ interaction. MSE + fMP4 for dashboards displaying many simultaneous camera feeds (e.g., 16–64 cameras), where scalability and browser efficiency matter more than achieving sub-200 ms latency. HLS only for archived playback and seeking. MJPEG only for lightweight previews or fallback scenarios. what happens if I use webRTC for displaying many simultaneous camera feeds (e.g., 16–64 cameras)?

You **can** use WebRTC for 16–64 cameras. It's not that you *can't*—it's that it often becomes the wrong engineering tradeoff.

Here's why.

Factor	WebRTC	MSE + fMP4
Latency	□ □ □ □ (100–300 ms)	□ □ □ □ (500 ms–2 s)
Single-camera experience	□ □ □ □ □	□ □ □ □
16–64 camera dashboard	□ □	□ □ □ □ □
Browser resource usage	High	Lower
Signaling complexity	High	Low
Scalability	Moderate	Excellent

Why WebRTC struggles with many streams

Each camera is typically its own peer connection and decoder.

For 32 cameras, the browser may be handling:

- 32 RTP streams
- 32 jitter buffers
- 32 congestion controllers
- 32 decoders
- 32 DTLS/SRTP sessions
- 32 packet-loss recovery loops

That's a lot of per-stream overhead before your application even starts rendering.

Browser decoding limits

Modern GPUs have hardware video decoders, but they are **not unlimited**.

For example:

- $1 \times 4K$ stream $\rightarrow$ easy
- $4 \times 1080p$ streams $\rightarrow$ easy
- $16 \times 1080p$ streams $\rightarrow$ depends on GPU
- $64 \times 1080p$ streams $\rightarrow$ generally unrealistic on a typical client

Even if decoding is hardware-accelerated, rendering 64 videos at 30 FPS is expensive.

Bandwidth

Assume each stream is:

- 1080p
- H.264
- 2 Mbps

Then:

- 16 cameras $\approx$ 32 Mbps
- 32 cameras $\approx$ 64 Mbps
- 64 cameras $\approx$ 128 Mbps

Even if the network can carry it, the browser still has to receive, decrypt, decode, and paint every frame.

Dashboard reality

When an operator views a 64-camera wall, they usually don't need:

- 30 FPS
- 150 ms latency
- full resolution

They mainly want situational awareness.

Many enterprise VMS therefore send:

- 5–10 FPS
- 360p–720p

- slightly higher latency

When the user double-clicks one camera, that stream is upgraded to high quality and low latency.

A hybrid architecture (what I'd recommend for your platform)

```
64-camera grid
  |
  ▼
MSE + fMP4
(5-10 FPS, lower resolution)

↓

User clicks Camera 17

↓

Switch Camera 17

↓

WebRTC
1080p
30 FPS
Ultra-low latency
PTZ enabled
```

This is how many professional VMS systems optimize both scalability and user experience.

For your CV platform

Given the reusable platform you're designing, I'd architect the streaming layer to support **multiple delivery protocols from the same decoded pipeline:**

```
RTSP
  |
  ↓
NVDEC
  |
  ↓
DeepStream
  |
  ↓
Inference + Metadata
├── WebRTC output (live inspection, PTZ)
├── MSE output (camera wall)
├── HLS output (playback)
└── JPEG snapshots
```

The AI pipeline runs **once**. Only the delivery protocol changes based on the frontend use case. This avoids duplicating inference work while letting each client receive the most appropriate

stream. This architecture also fits well with your goal of building a reusable computer vision platform that can support many different surveillance and industrial applications.

rendering 64 videos at 30 FPS is expensive